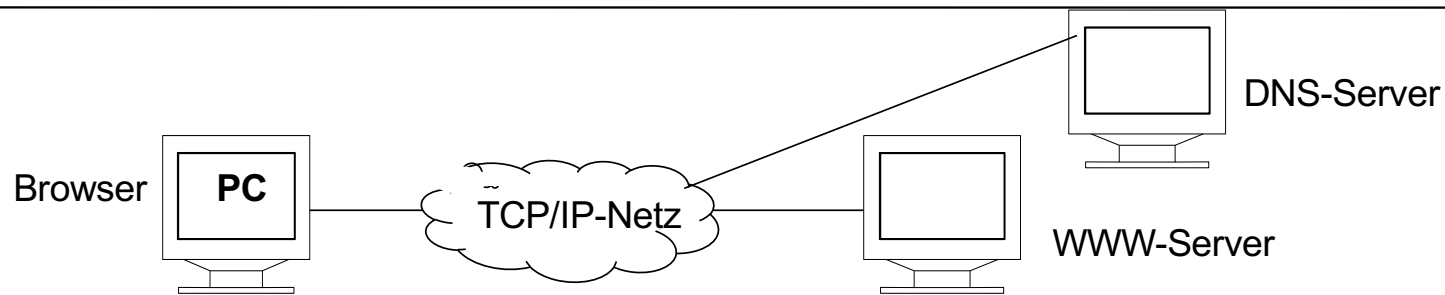




## Abruf von Webseiten (ohne Persistent-Connections)



Browser fragt DNS nach der IP-Adresse des Servers

DNS antwortet

Browser öffnet eine TCP-Verbindung (3-Way-Handshake) zu Port 80 des Rechners

Browser sendet das Kommando GET xyz

WWW-Server schickt die angeforderte Ressource zurück

Verbindung wird wieder abgebaut.



# HTTP

## Binärdaten und das textbasiert?

### HTTP wurde textbasiert konzipiert (eMail auch!)

#### Wie können binäre Daten übertragen werden?

- Die Antworten des Servers auf eine vollständige GET-Request beinhaltet MIME-Informationen
- MIME = Multipurpose Internet Mail Extensions
- Definiert die Kodierungsregeln für Nicht-ASCII-Nachrichten
- MIME ermöglicht die Nutzung verschiedener Kodierungen (media types) in einer Nachricht

#### Die “Content-Type:”-Zeile im MIME-Header legt den Datentyp (type/subtype) einer Nachricht fest

#### Content-Transfer-Encoding: definiert die Transfersyntax, in der die Daten des Hauptteils übertragen werden, wird aber bei HTTP nicht benutzt

- Content-Encoding und Transfer-Encoding Felder

#### Beispiele:

- Content-Type: text/html
- Content-Type: image/GIF



# HTTP

## Binärdaten und das textbasiert

```
MIME-Version: 1.0
Content-Type: MULTIPART/MIXED;
  BOUNDARY= "8323328-2120168431-824156555=:325"
--8323328-2120168431-824156555=:325
Content-Type: TEXT/PLAIN; charset=US-ASCII
A picture is in the appendix
--8323328-2120168431-824156555=:325
Content-Type: IMAGE/JPEG; name="picture.jpg"
Content-Transfer-Encoding: BASE64
Content-ID: <PINE.LNX.3.91.960212212235.325B@localhost>
Content-Description:
/9j/4AAQSkZJRgABAQEAlgCWAAD/2wBDAAEBAQEBAQEBAQEBAQEBAQIBAQEBA
QIBAQECAGICAgICAgIDAwQDAwMDAwICAwQDAwQEBAQEAgMFBQQEBQQEBAT/
2wBDAQEBAQEBAQIBAQIEAwIDBAQEBA [...]
KKACiigAooooAKKKKACiigAooooAKKKKACiigAooooAKKKKACiigAoooo
AKKKKACiigAooooAKKKKACiigAooooAKKKKACiigAooooAKKKKACiig
AooooAD//Z
---8323328-2120168431-824156555=:325 --
```



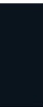
## (No-IP-Based) Virtual Hosts

---

- Auf einem Rechner sollen verschiedene Domains und Web-Server zur Verfügung stehen
- Ein oder mehrere Webserver (Software) sollen die Anfragen, für die auf dem Rechner vorhandenen Domains, beantworten
- Typische Anwendung: Web-Hosting (Provider)
- Nach der DNS-Auflösung geht die Information, mit welchen Servern man eigentlich sprechen wollte, verloren
- Anwendungsebene liefert die Information erneut mit

### Merke:

- Die Namensauflösung FQDN-IP ist nicht injektiv. Es gibt also beliebig viele Domänen, die faktisch an die gleiche Zone gehen.
- Wenn firma1.de von provider1.de bedient wird, so wird für firma1.de in der .de-Zone typischerweise ein NS-Record hinterlegt, der aussagt welche autoritativen Nameserver für firma1.de zuständig sind, z. B. ns1.provider1.de

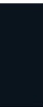




# Absicherung der Kommunikation: TLS/https

---

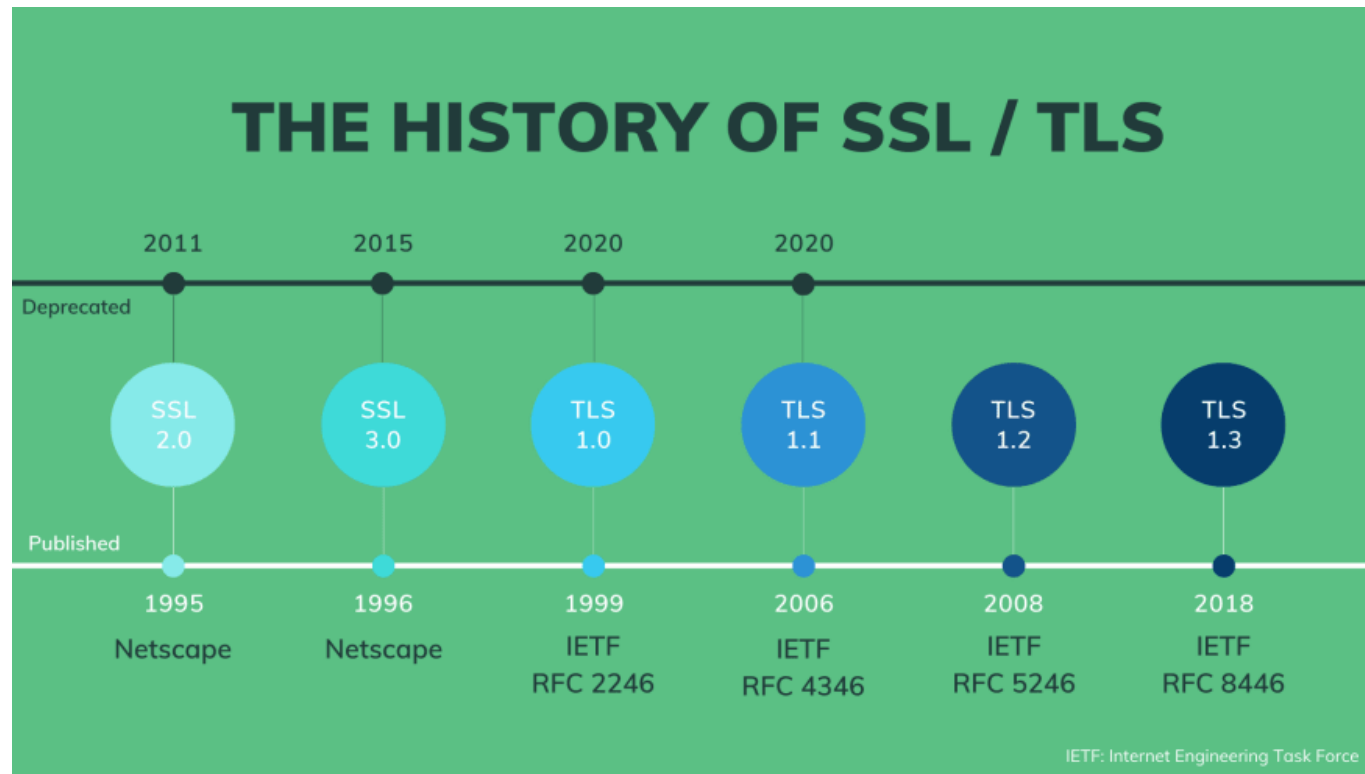
- Kommunikation kann durch Einsatz des Secure-Socket-Layer-Protokolls (SSL) abgesichert werden
  - SSL wurde von der Firma Netscape entwickelt und ist unter dem Namen **Transport Layer Security (TLS)** standardisiert (RFC2246)
  - TLS/SSL sichert die Transportschicht ab – Basis ist das RSA-Verfahren
  - TLS/SSL ist verbindungsorientiert
- Das Secure Socket Layer-Protokoll ist ein hat drei grundlegende Eigenschaften
  1. Eine Verbindung ist privat. Beim Verbindungsaufbau wird ein **Sitzungsschlüssel** bestimmt, der dann zur symmetrischen Verschlüsselung benutzt werden kann
  2. Die beiderseitige Authentifikation (mutual authentication) wird (durch asymmetrische Verschlüsselung) ermöglicht
  3. Eine spätere Nachricht wird immer durch einen Hash gegen Veränderungen abgesichert





# TLS: Die Historie

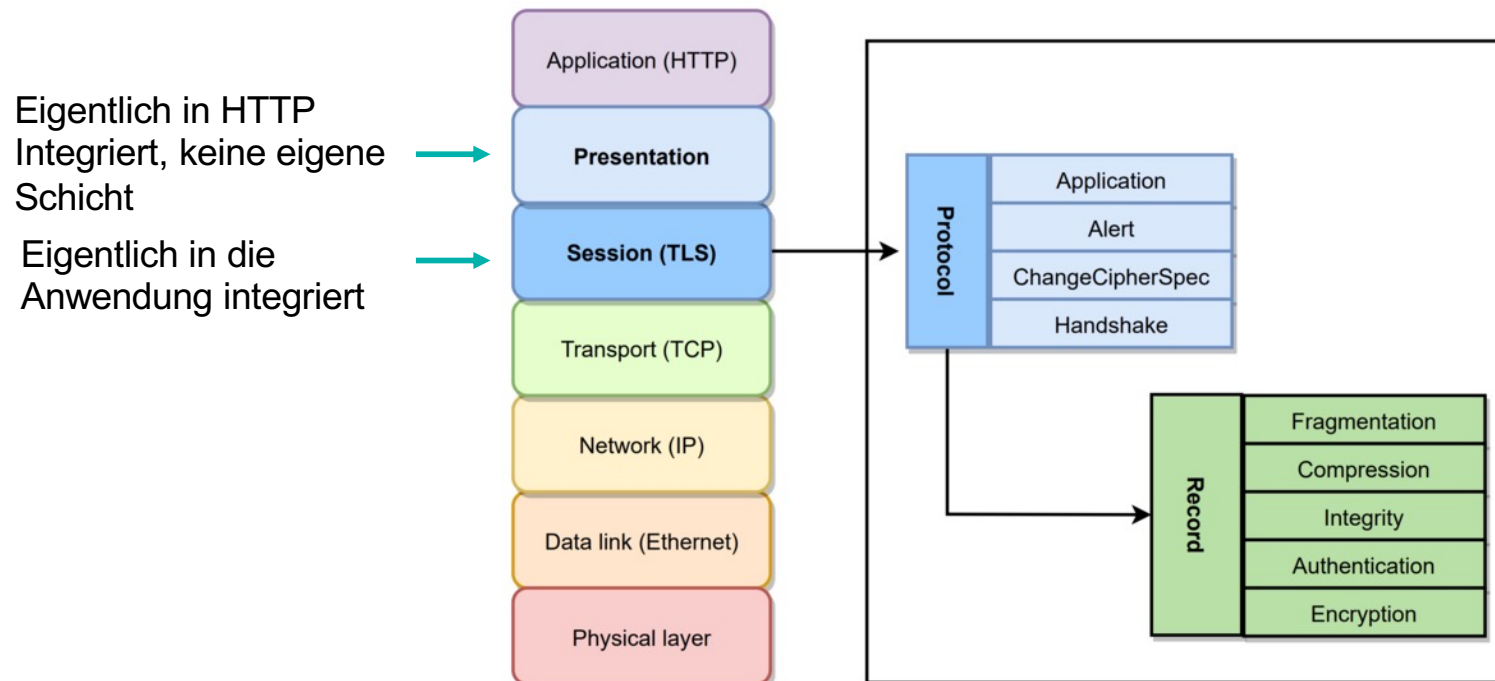
<https://dev.to/techschoolguru/a-complete-overview-of-ssl-tls-and-its-cryptographic-system-36pd>





## SSL im ISO-OSI - Stack

- Transparente Schicht zwischen TCP/IP und Anwendung
- Universell einsetzbar zur Verschlüsselung der Inhaltsdaten



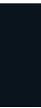
[https://commons.wikimedia.org/wiki/File:TLS\\_in\\_OSI.png](https://commons.wikimedia.org/wiki/File:TLS_in_OSI.png)



## TLS – Handshake Protokoll

---

- Das TLS-Handshake-Protokoll erledigt folgende Aufgaben:
  - Ermittlung des stärksten gemeinsam unterstützten Algorithmus
  - Optionale Authentifikation der Kommunikationspartner (seltener wird der Client Authentifiziert)
    - > Hierbei kommen Zertifikate zum Einsatz
    - > Konkret werden die Zertifikatsketten bis zum selbst-signierten Zertifikat genutzt
  - Bestimmung eines Session Keys zur symmetrischen Verschlüsselung
  - Mit der Fähigkeit den Session Key abzuleiten ist der Nachweis der Kenntnis des Private Keys erbracht worden. Ob damit die Authentizität gesichert ist hängt von dem Vertrauen in die Zertifikatskette ab







# Warum TLS

<https://dev.to/techschoolguru/a-complete-overview-of-ssl-tls-and-its-cryptog>

## WHY TLS?



### Authentication

Verify the identity of the communicating parties with asymmetric cryptography



### Confidentiality

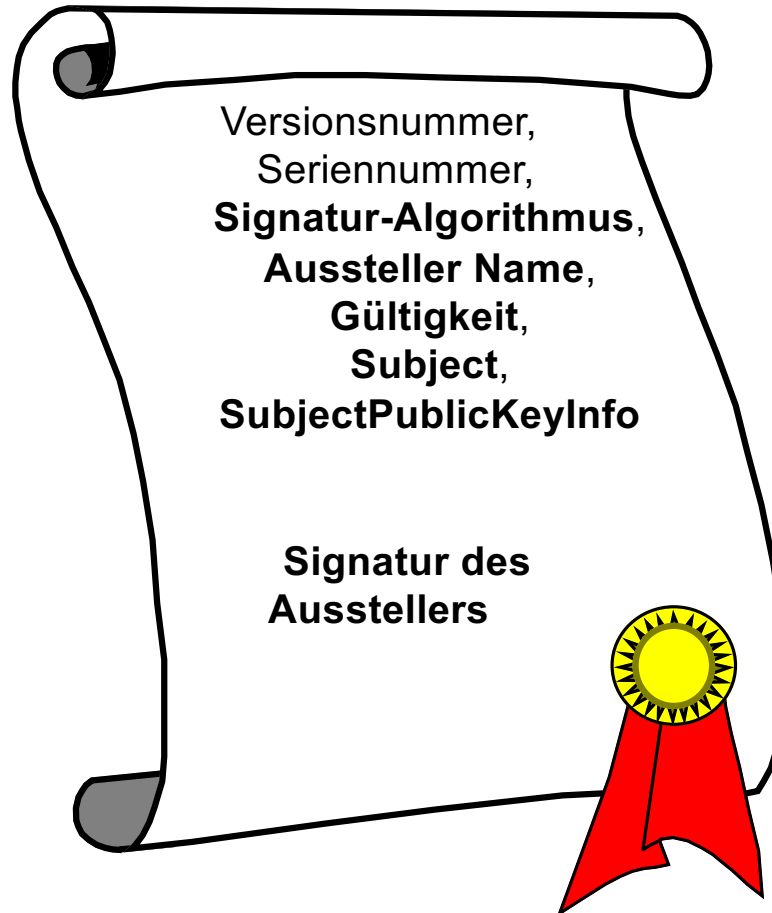
Protect the exchanged data from unauthorized access with symmetric cryptography



### Integrity

Prevent alteration of data during transmission with message authentication code

## X509 Zertifikate (2)



### Idee:

`{{Klartext}private_key}public_key =`  
`{{Klartext}public_key}private_key =`  
`Klartext`

## X509 Zertifikate

**Aussteller der  
die Identität  
validiert hat**

**Die zu  
prüfende  
Identität**

**Der Aussteller  
bestätigt die  
zuordnung des  
Public Key**

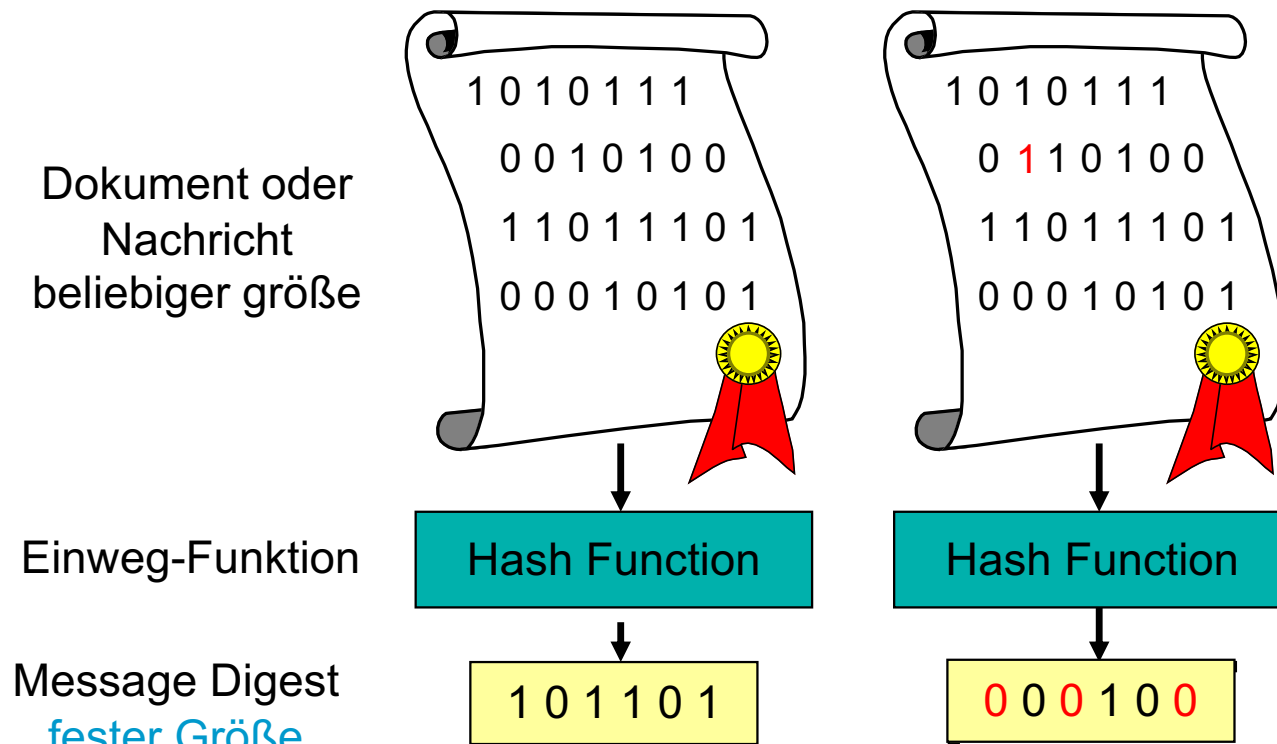
Versionsnummer,  
Seriennummer,  
**Signatur-Algorithmus,**  
**Aussteller,**  
**Gültigkeit,**  
Subject,  
**SubjectPublicKeyInfo**

**Signatur des  
Ausstellers**

**{MD5 hash}**  
Mit dem  
private key  
des  
Ausstellers  
**Verschlüsselter  
Hash**



# Signaturen: Einweg-Hash-Funktionen



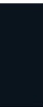
- Die Änderung eines einzelnen Bits im Dokument sollte ungefähr 50% der Hash-bits verändern!



## Der klassische TLS Handshake (vereinfacht vor V1.3)

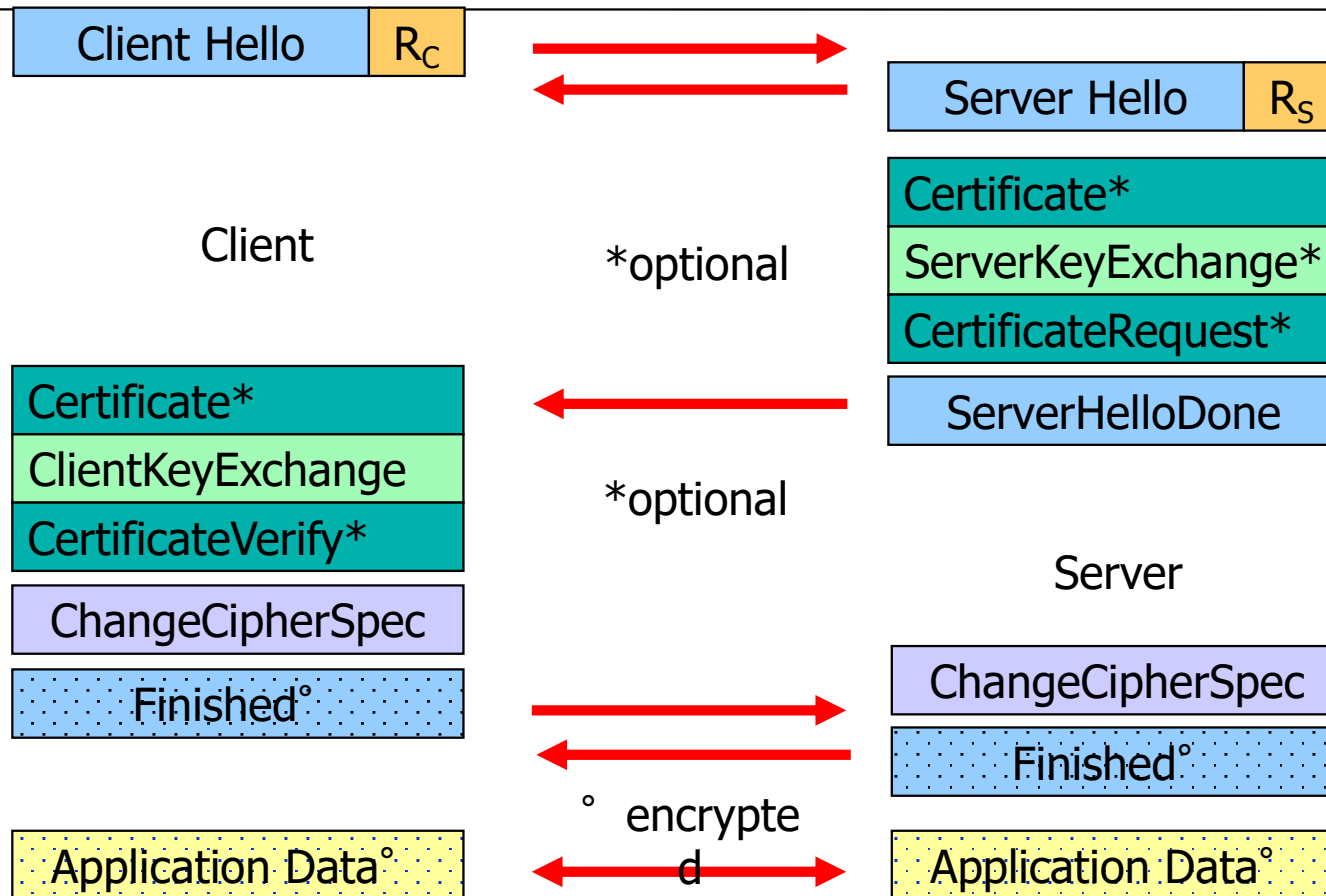
---

- Client-Hello-Nachricht:
  - Unterstützte Algorithmen, Zufallsnummer des Clients
  - Session ID (**Abkürzung des Handshakes somit möglich**)
- Server-Hello:
  - Ausgewählter Algorithmus, **Zufallsnummer** des Servers
  - Session ID
- Server **Certificates**
- Server Hello Done
- Client Key Exchange
  - Nach Prüfung des Zertifikats wird **mit dem Public Key des Servers** die Basis des **Sessions Keys verschlüsselt**
- Optional Client Zertifikat + mit Private Key verschlüsselte **Zufallsnummer**
- Beiderseitiges Finished





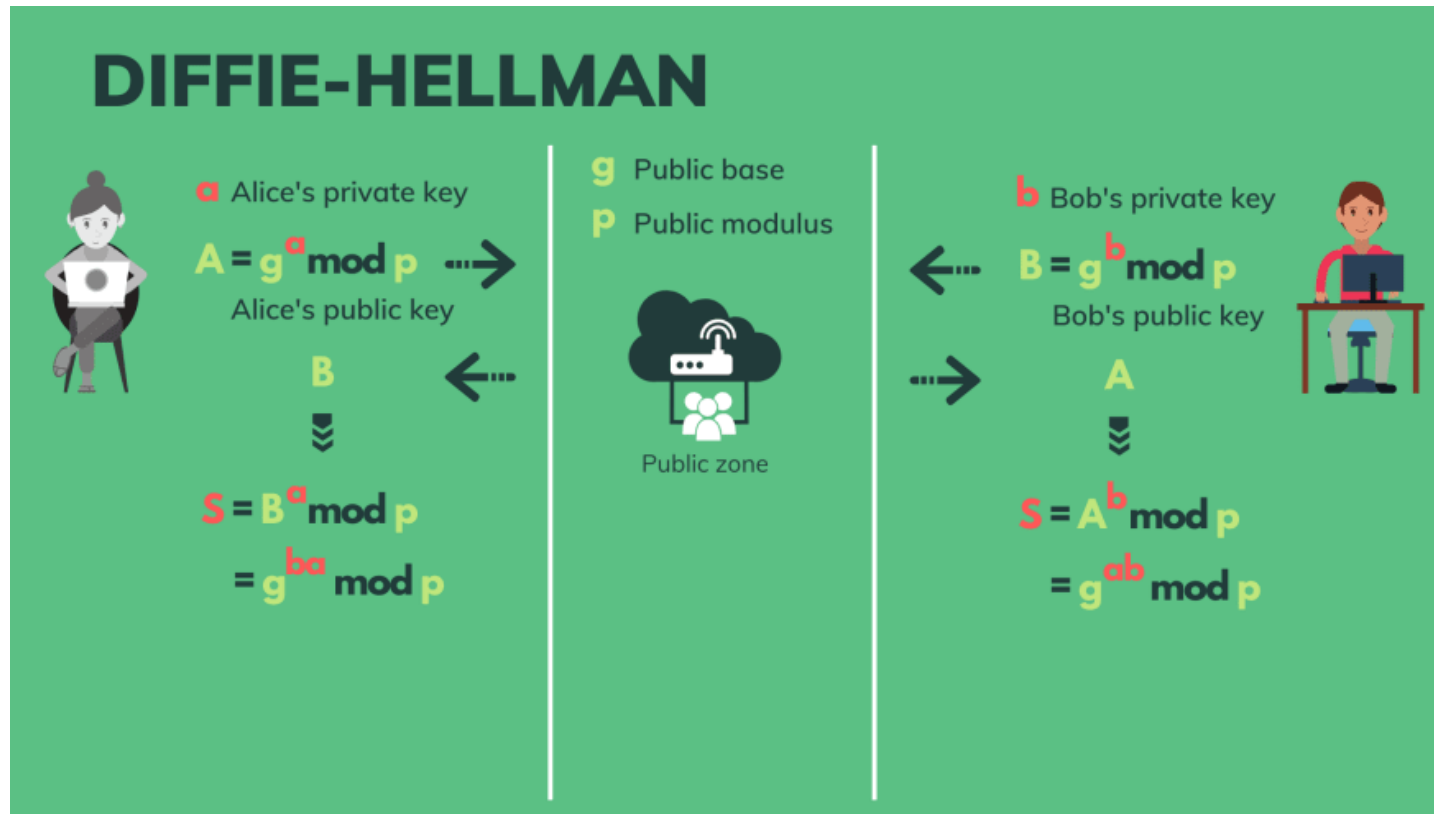
# TLS Handshake Protocol





# Der Diffie-Hellman-Schritt

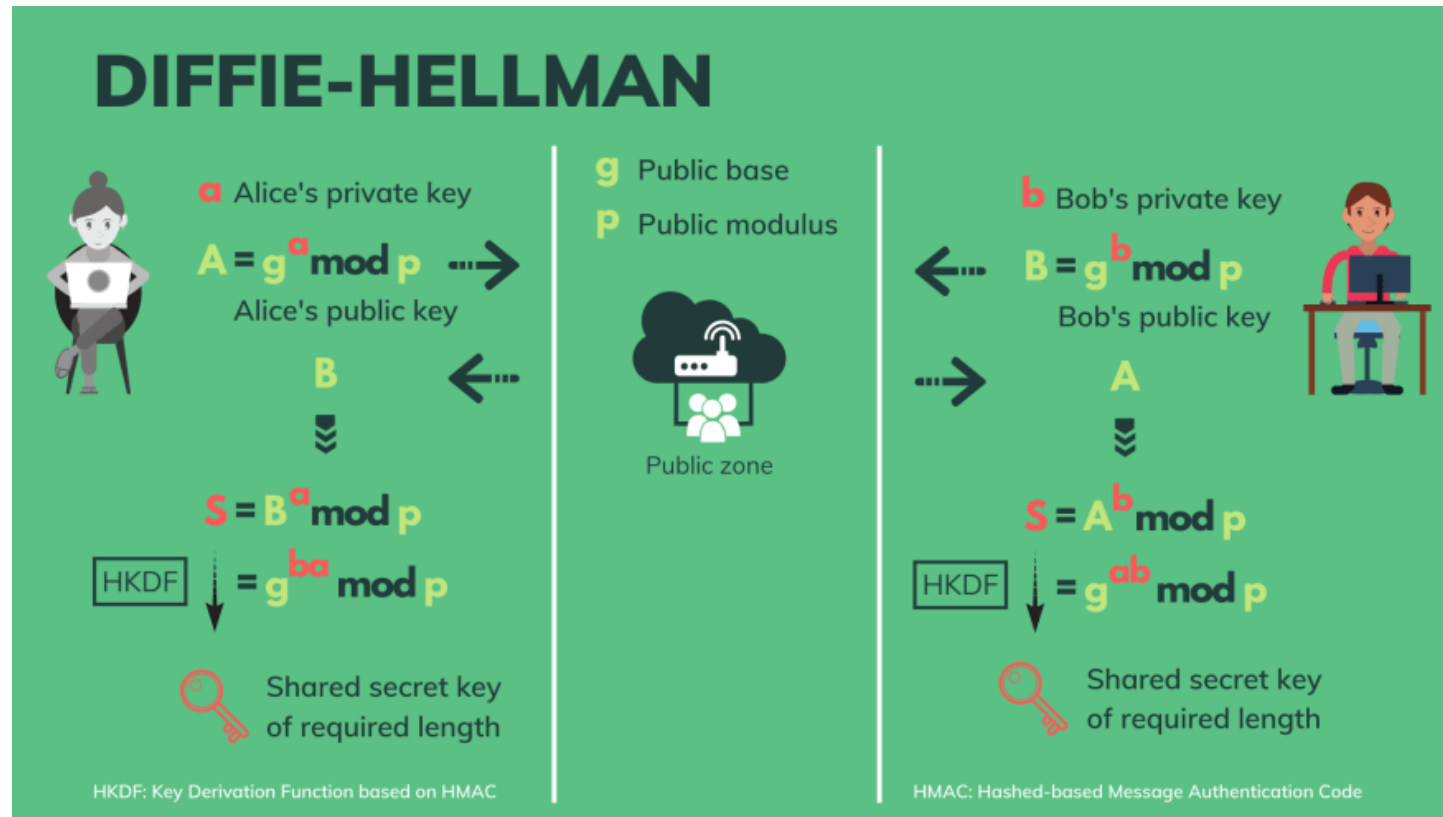
<https://dev.to/techschoolguru/a-complete-overview-of-ssl-tls-and-its-cryptog>





# Der Diffie-Hellman-Schritt

<https://dev.to/techschoolguru/a-complete-overview-of-ssl-tls-and-its-cryptog>





# Der Diffie-Hellman-Schritt

<https://dev.to/techschoolguru/a-complete-overview-of-ssl-tls-and-its-cryptog>

## IS DIFFIE-HELLMAN SECURE?



$p, g, A, B$

$$A = g^a \bmod p$$

$$B = g^b \bmod p$$



$\leadsto a, b, S?$



### Trapdoor function

$\left\{ \begin{array}{l} p : 2048\text{-bit prime number} \\ a, b : 256\text{-bit random integers} \\ g : \text{primitive root modulo } p \end{array} \right.$

Given  $a, p, g \leadsto$  easy to compute  $A = g^a \bmod p \leadsto$  Fast:  $O(\log(a))$

Given  $A, p, g \leadsto$  hard to compute  $a \leadsto$  Discrete logarithm problem

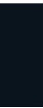
$\leadsto$  Very long time to solve

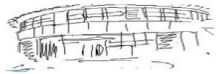


## TLS Record Layer

---

- Vollständig getrennt vom Handshake Protokoll
- Verschickt Daten symmetrisch mit dem im Handshake ausgehandelten Verschlüsselungsalgorithmus, verschlüsselt mit dem symmetrischen Session Key
- Bildet zu jedem Datenblock einen Hash (Message Digest) zur Sicherung der Integrität





# Quick verbirgt mehr

HTTP w/ TLS + TCP

|                        |         |                  |        |
|------------------------|---------|------------------|--------|
| source port            |         | destination port |        |
| sequence number        |         |                  |        |
| acknowledgement number |         |                  |        |
| hlen                   | flags   |                  | window |
| checksum               |         | urgent pointer   |        |
| [options]              |         |                  |        |
| type                   | version |                  | length |
| length                 |         |                  |        |
|                        |         |                  |        |

HTTP w/ QUIC

|             |  |                      |  |
|-------------|--|----------------------|--|
| source port |  | destination port     |  |
| length      |  | checksum             |  |
| 01S         |  | [dest connection id] |  |
|             |  |                      |  |

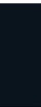
<https://www.youtube.com/watch?v=31J8PoLW9iM&t=9104s>



# FTP - File Transfer Protocol

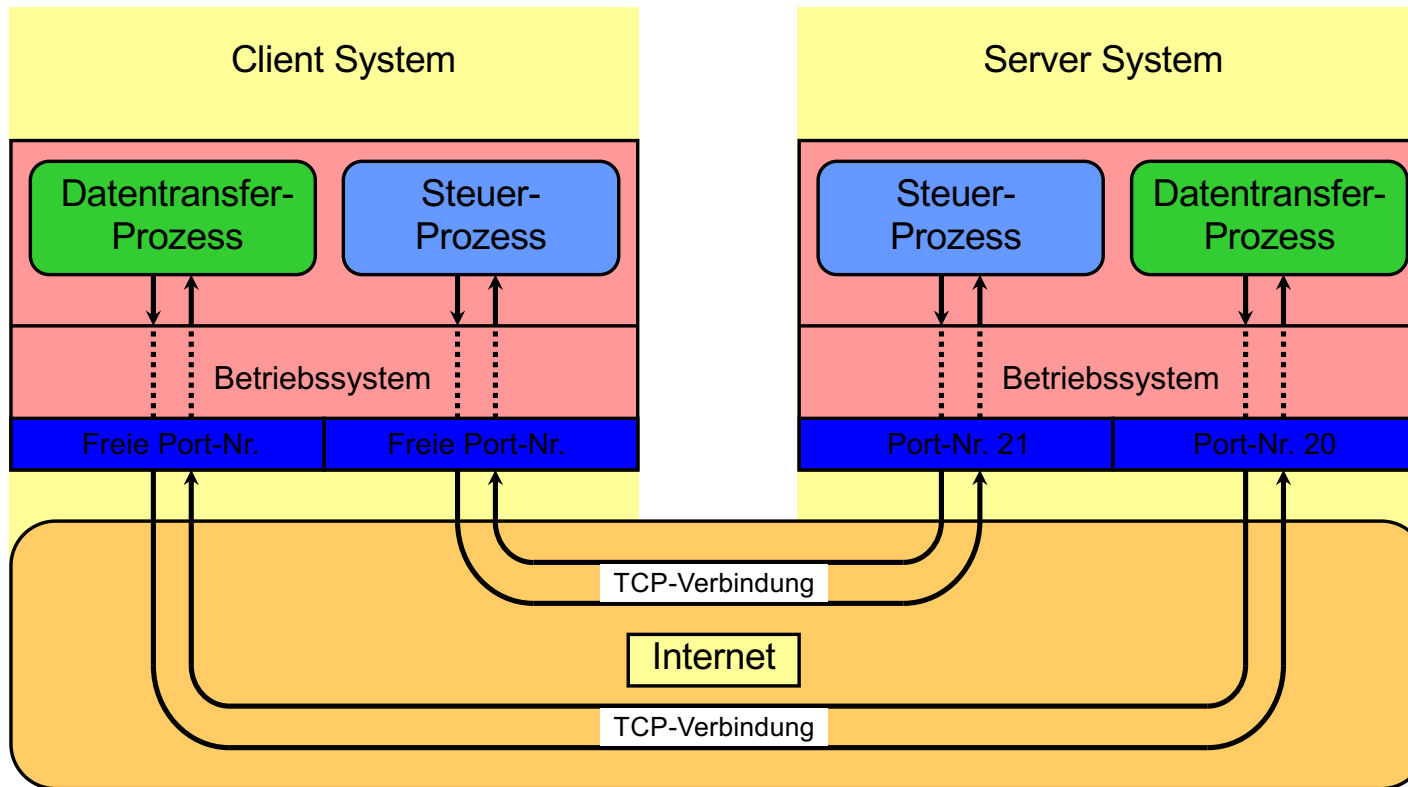
---

- FTP ist der *Internet-Standard für die Übertragung von Dateien*
- FTP wird benutzt, um eine **komplette Datei** von einem Rechner auf einen anderen zu **kopieren**
- FTP bietet neben dem reinen File-Transfer noch andere Möglichkeiten:
  - **Interaktiver Zugriff** durch den Nutzer (z.B. Wechsel von Verzeichnissen)
  - **Format-Spezifikation** (Binär- oder Textdateien, ASCII- oder EBCDIC-Code)
  - **Authentifizierung** urspr. nicht empfehlenswert : (login-Name und Passwort)
  - Es gibt Varianten, die eine verbesserte Authentifikation vornehmen





# FTP Client/Server-Beziehung





## FTP: Passive und Active Mode

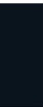
---

**FTP ist deshalb besonders**, weil es für die Datenübertragung eine separate TCP-Verbindung verwendet. Diese kann somit über verbesserte Parameter (WSCALE-Option verfügen).

Es gibt 2 verschiedene Arten, die Datenverbindung zu öffnen:

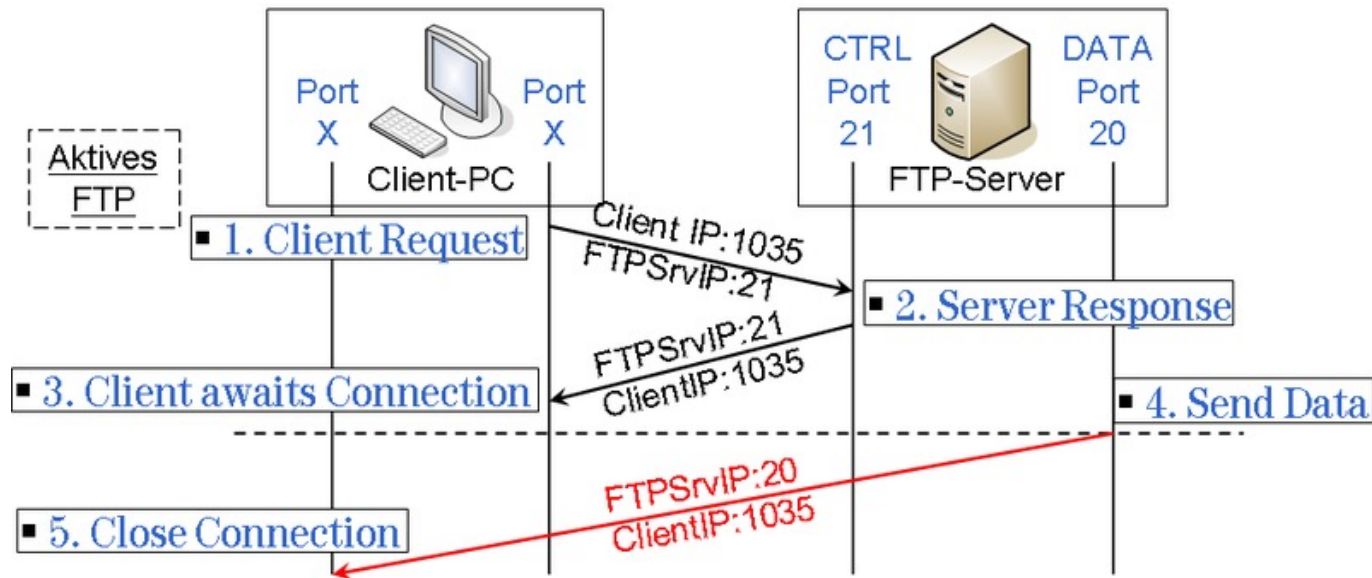
1. **Active Mode:** Hier horcht der Client für die Datenverbindung auf einem zufälligen Port und teilt diesen dem Server über die Kontrollverbindung mit. Hierzu dient das PORT-Kommando. Der Server (Port 20) verbindet sich dann mit dem Client und wickelt den Transfer hierüber ab
2. **Passive Mode:** Öffnet der Server einen Port und teilt diesen dem Client mit. Der Client verbindet sich nun durch mit dem Server. Das PASV-Kommando regelt dies. Vorteil: **Dieses Verfahren funktioniert auch, wenn der Client Adressumsetzungen (NAT) oder Firewalls einsetzt**

**Merke:** FTP ist ein Protokoll, welches 2 Verbindungen nutzt: einen eher interaktiven Kontrollkanal und einen durchsatzoptimierten Transferkanal





# FTP: Active Mode

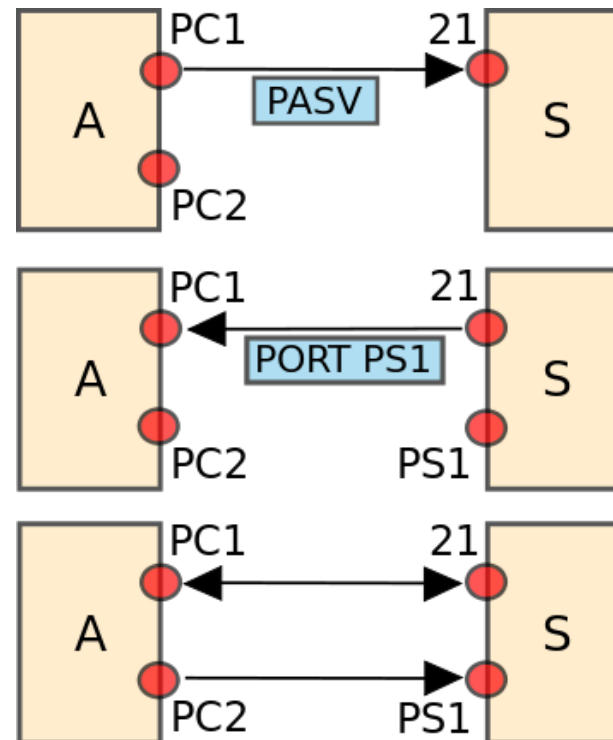


Attribution: [Mdjango](#)

<https://commons.wikimedia.org/wiki/File:AktivesFTP.png>



## FTP: Passive Mode



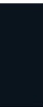
[https://commons.wikimedia.org/wiki/File:Passive\\_FTP\\_Verbindung.svg](https://commons.wikimedia.org/wiki/File:Passive_FTP_Verbindung.svg)

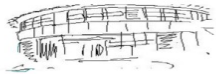




## FTP - Befehle

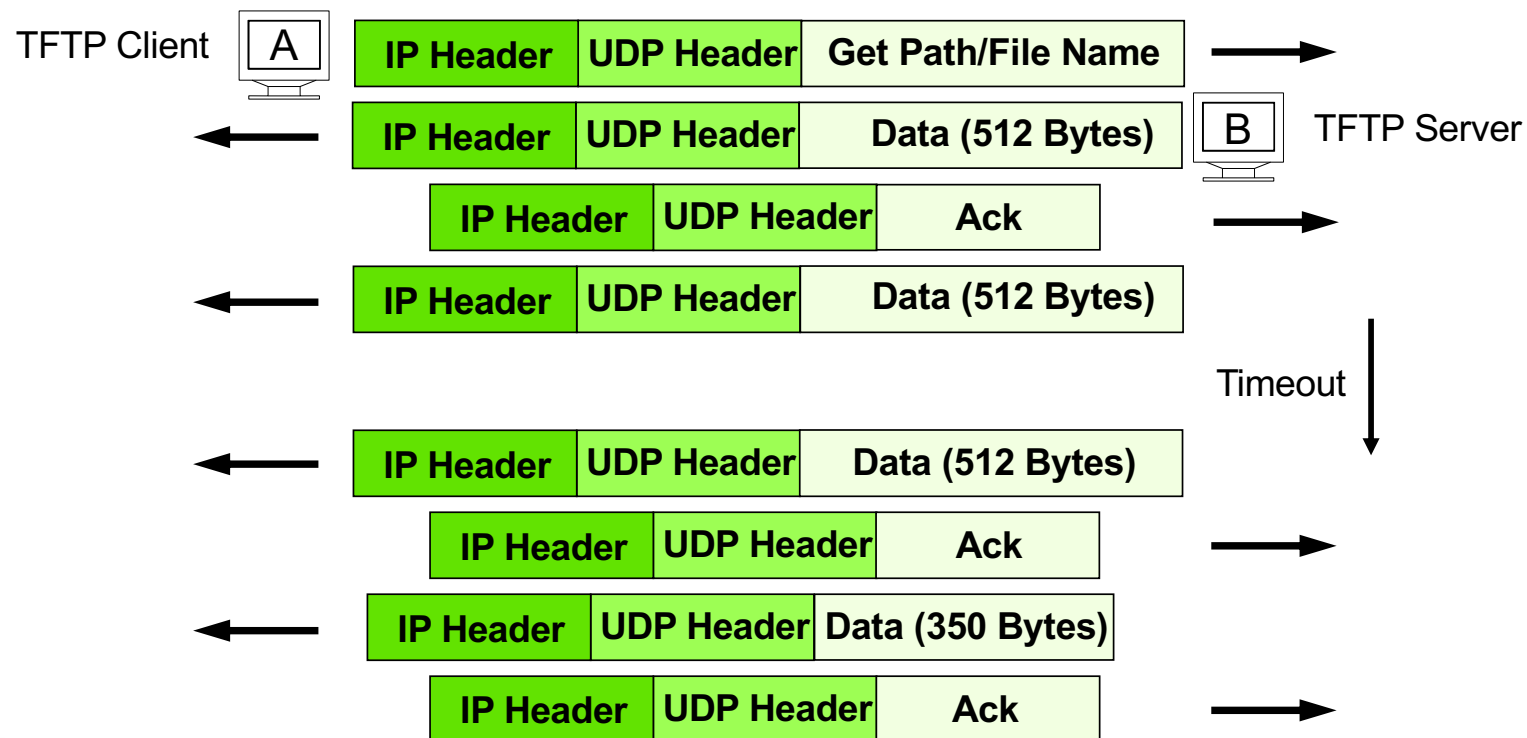
| Kommando   | Wirkung                                               |
|------------|-------------------------------------------------------|
| open       | Verbinden zum FTP-Server                              |
| disconnect | Beende die FTP-Sitzung                                |
| user       | Sende Benutzerinformationen nach dem Verbinden        |
| cd         | change directory auf dem entfernten Rechner           |
| lcd        | change directory auf dem eigenen Rechner              |
| pwd        | Drucke das Arbeitsverzeichnis des entfernten Rechners |
| get/mget   | Der Client empfängt ein (bzw. mehrere) Dokument       |
| put/mput   | Der Client sendet ein (bzw. mehrere) Dokument         |
| binary     | Setze den Übertragungsmodus auf binary                |
| ascii      | Setze den Übertragungsmodus auf ASCII                 |
| dir/ls     | Liste den Inhalt des entfernten Verzeichnisses auf    |
| help       | Hilfe                                                 |
| delete     | Lösche eine entfernte Datei                           |
| bye        | Beende die FTP-Sitzung, Abbruch                       |





# TFTP - Trivial File Transfer Protocol

- TFTP ist ein sehr **einfaches** Protokoll für den File-Transfer (EEPROM!)
- die Kommunikation läuft über Port 69 und benutzt **UDP**, nicht TCP
- TFTP hat **keine Authentifizierung**
- TFTP benutzt immer 512-Byte-Blöcke



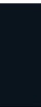


## Elektronische Post: E-Mail

---

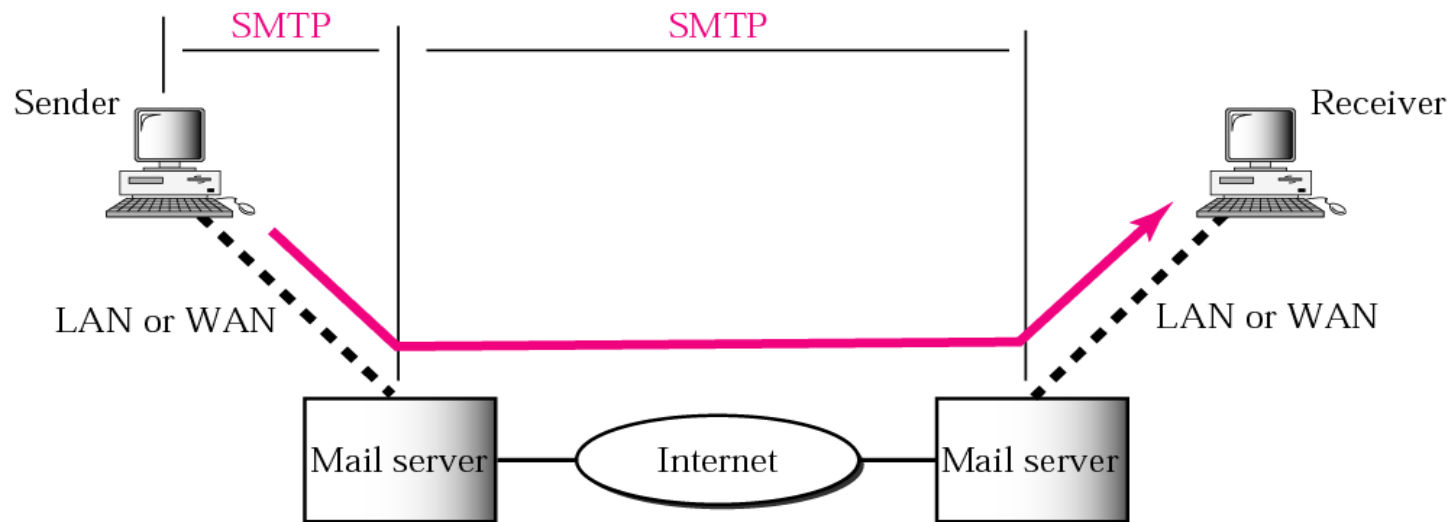
Ein E-Mail-System besteht im allgemeinen aus zwei Subsystemen:

- **User Agent** (UA, normales Email-Programm)
  - läuft meist auf dem Rechner des Benutzers und hilft bei der Bearbeitung von E-Mails
  - Erstellung neuer und Beantwortung alter E-Mail
  - Empfang und Anzeigen von E-Mail
  - Verwaltung von erhaltener E-Mail
- **Message Transfer Agent** (MTA, Mailserver, mailrelay)
  - läuft meist im Hintergrund (rund um die Uhr)
  - Zustellung von E-Mails, die von User Agents losgeschickt wird
  - Zwischenspeicherung von Nachrichten für User oder andere Message Transfer Agents
- **Simple Mail Transfer Protokoll (SMTP)** zum Verschicken von E-Mails





# Elektronische Post: E-Mail





# Das Senden von E-Mails

---

Zum Verschicken einer eMail muss der Benutzer folgende Angaben machen:

- *Nachricht* (meist normaler Text + Attachements, z.B. Word-Datei, GIF...)
- *Zieladresse* (i.a. in der Form mailbox@location, z.B. v.sander@fh-aachen.de)
- evtl. *zusätzliche Parameter* bzgl. Priorität oder Sicherheit

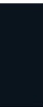
## eMail-Formate

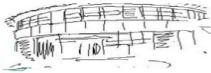
Zwei verbreitete Standards:

- **RFC 822** (ARPA Internet Text Messages)
- **MIME** (Multipurpose Internet Mail Extensions)

Bei **RFC 822** besteht die E-Mail aus

- einem einfachen “Umschlag” (erstellt durch den Message Transfer Agent anhand der Daten im Header),
- einer Reihe von Header-Feldern (je eine Zeile ASCII-Text),
- einer Leerzeile und
- der eigentlichen Nachricht (Message Body).

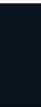


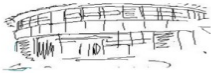


## Header einer eMail

| Header       | Bedeutung                                                                          |
|--------------|------------------------------------------------------------------------------------|
| To:          | eMail-Adresse des Hauptempfängers (evtl. mehrere oder auch Verteilerliste)         |
| Cc:          | Carbon Copy (Durchschrift), Email-Adressen von weniger wichtigen Empfängern        |
| Bcc:         | Blind Carbon Copy, Empfänger, die anderen Empfängern <i>nicht</i> angezeigt werden |
| From:        | Person, die die Nachricht generiert hat                                            |
| Sender:      | Adresse des eigentlichen Senders der Nachricht (evtl. ungleich der "From-Person")  |
| Received:    | Je ein Eintrag pro Message Transfer Agent auf dem Weg zum Ziel                     |
| Return-Path: | Pfad zurück zum Sender (meist nur eMail-Adresse des Senders)                       |
| Date:        | Sende-Datum und -Uhrzeit                                                           |
| Reply-To:    | eMail-Adresse, an die Antworten gerichtet werden sollen                            |
| Message-Id:  | Eindeutige Nummer der Email (für spätere Referenzen)                               |
| In-Reply-To: | Message-Id der Nachricht, auf die geantwortet wurde                                |
| References:  | Andere relevante Message-Ids                                                       |
| Subject:     | Einzeilige Angabe des Inhalts der Nachricht (wird beim Empfänger angezeigt)        |

*Anmerkung: neben dieser Liste existieren noch weitere mögliche Header-Felder*





## Header einer E-Mail: MIME

RFC 2822 *nur geeignet für Nachrichten aus reinem ASCII-Text ohne Sonderzeichen.*

Heutzutage zusätzlich gefordert:

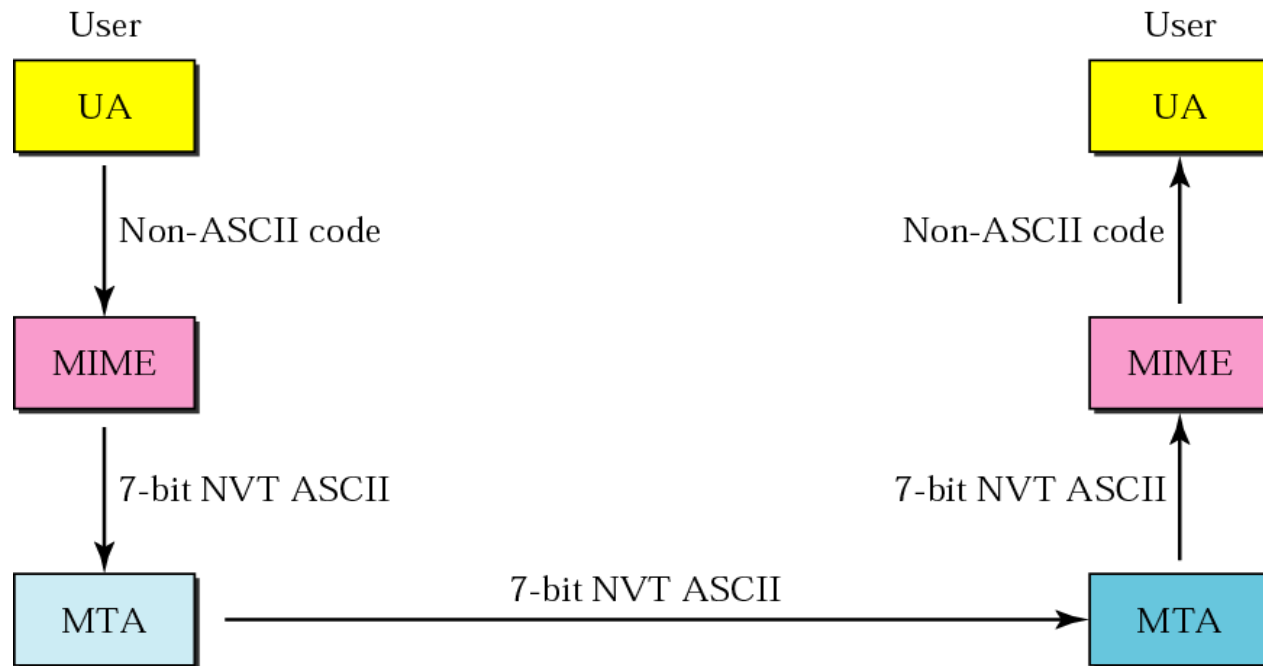
- eMail in Sprachen mit Sonderzeichen (z.B. französisch oder deutsch)
- eMail in Sprachen, die nicht das lateinische Alphabet benutzen (z.B. russisch)
- eMail in Sprachen, die überhaupt kein Alphabet benutzen (z.B. japanisch)
- eMail, die teilweise überhaupt keinen Text enthält (z.B. Audio oder Video)

**MIME** behält das RFC 2822 Format bei, definiert dabei aber eine Struktur im Message Body (durch zusätzliche Header) und Kodierungsregeln für Nicht-ASCII-Zeichen

| Header                     | Bedeutung                                                                                                                                   |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| MIME-Version:              | Kennzeichnet die benutzte Version von MIME                                                                                                  |
| Content-Description:       | Für Menschen lesbarer String, der den Inhalt der Nachricht beschreibt                                                                       |
| Content-Id:                | Eindeutiger Bezeichner des Inhalts                                                                                                          |
| Content-Transfer-Encoding: | Verpackung, die für den Inhalt der eMail gewählt wurde (manche Netze verstehen z.B. nur ASCII-Zeichen). Beispiele: base64, quoted-printable |
| Content-Type:              | Typ/Subtyp gemäß RFC 1521, z.B. text/plain, image/jpeg, multipart/mixed                                                                     |



# MIME-Ablauf



Der konkrete Ablauf wird nunmehr in den RFCs 2045 (MIME) und 2821 (SMTP) beschrieben





# eMail über POP3 und SMTP

---

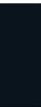
## Simple Mail Transfer Protocol (SMTP)

- Versenden von eMails über TCP-Verbindung (Port 25)
- SMTP ist ein einfaches ASCII-Protokoll
- Ohne Prüfsummen, ohne Verschlüsselung
- Ist der Server zum Empfangen bereit, signalisiert er dies dem Client. Dieser sendet die Information, von wem die eMail kommt und wer der Empfänger ist. Ist der Empfänger dem Server bekannt, sendet der Client die Nachricht, der Server bestätigt den Empfang.

## Post Office Protocol Version 3 (POP3)

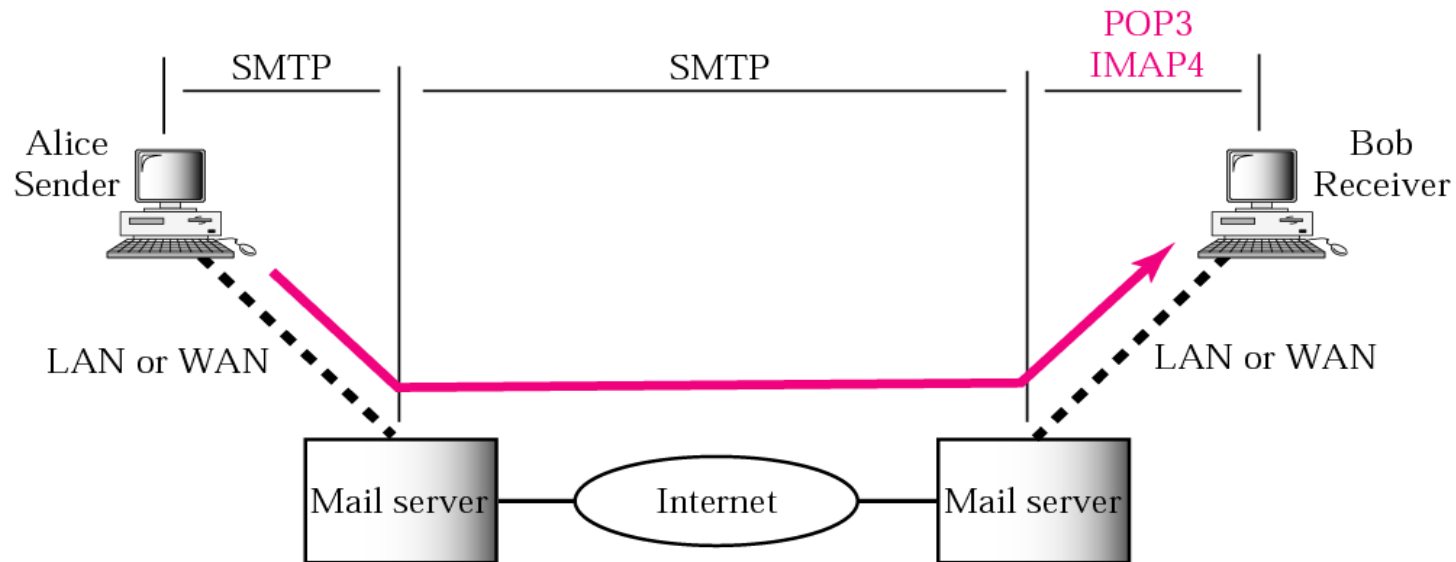
- Abholen der eMails beim Server über eine TCP-Verbindung, Port 110
- Befehle zum An- und Abmelden, Nachrichten herunterladen, Nachrichten auf dem Server löschen oder liegen lassen, Nachrichten ohne vorherige Übertragung vom Server direkt löschen

**IMAP (Interactive Mail Access Protocol).** Hier werden die eMails nicht abgerufen und lokal gespeichert, sondern bleiben auf dem Server liegen!





# Mails



Der DNS Mail Exchange Record teilt dem Mail-Server mit, welchen Mailserver die Zieldomäne hat



# SMTP - Befehlsabfolge

Kommunikation zwischen Partnern (von abc.com nach beta.edu) in Textform der Art:

|                                                                       |                                          |
|-----------------------------------------------------------------------|------------------------------------------|
| S: 220 <beta.edu> Service Ready                                       | /* Empfänger ist bereit */               |
| C: HELO <abc.com>                                                     | /* Identifikation des Senders */         |
| S: 250 <beta.edu> OK                                                  | /* Server meldet sich */                 |
| C: MAIL FROM:<Krogull@abc.com>                                        | /* Sender der eMail */                   |
| S: 250 OK                                                             | /* Senden ist erlaubt */                 |
| C: RCPT TO:<Bolke@beta.edu>                                           | /* Empfänger der eMail */                |
| S: 250 OK                                                             | /* Empfänger bekannt */                  |
| C: DATA                                                               | /* Jetzt kommen die Daten */             |
| S: 354 Start mail input; end with "<crLf>.<crLf>" on a line by itself |                                          |
| C: From: Krogull @ ..... <crLf>.<crLf>                                | /* ab hier normales Nachrichtenformat */ |
| S: 250 OK                                                             |                                          |
| C: QUIT                                                               | /* Beenden der Verbindung */             |
| S: 221 <beta.edu> Server Closing                                      |                                          |

S = Server, empfangender MTA / C = Client, sender MTA



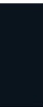
# IMAP

---

**Idee:** Zugriff auf Server-Verzeichnisse wie auf lokale Verzeichnisse

- Daten verbleiben auf dem Server. Alle Aktionen führt der Client durch
- Das Protokoll ist komplexer als bei POP3 und erlaubt die Manipulation von Mailverzeichnissen auf dem Server (Erstellen, Umbenennen, Löschen; Setzen/Löschen von Flags; Suchen von Mails)
- Reduziert die zu übertragenen Daten, da zunächst nur die Nachrichten-Header übertragen werden
- Offline-Betrieb und Resynchronisation mit dem Server möglich
- Spezifiziert in **RFC 3501**

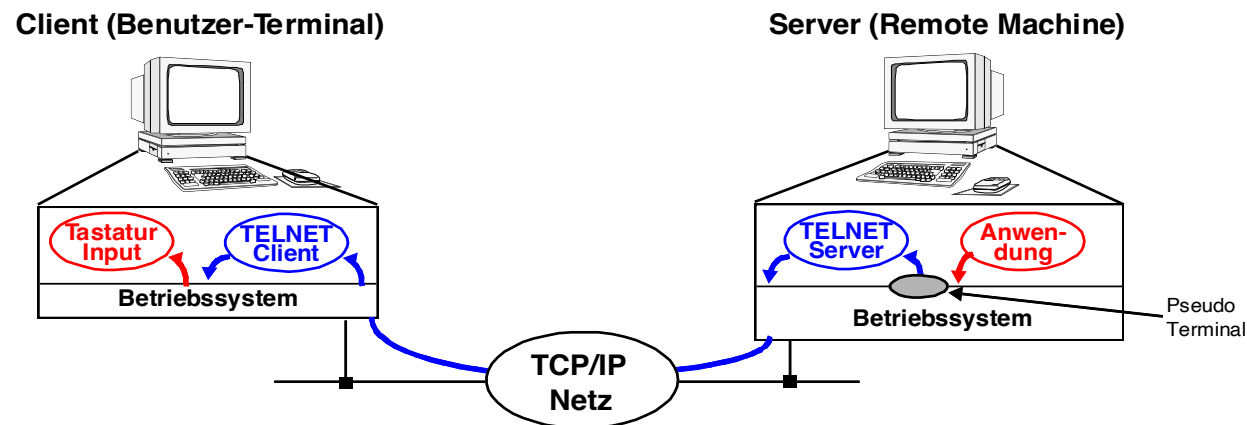
**Mit IMAP verbleiben die eMail auf dem Server**





# Telnet - Entferntes Arbeiten

- TCP ermöglicht den transparenten, interaktiven Gebrauch von „entfernten“ Maschinen
- verbreitetes Protokoll: TELNET, welches auf einer Client/Server-Kommunikation basiert
- Ein „Pseudo-Terminal“ des Servers interpretiert Zeichen, als kämen sie von der eigenen Tastatur



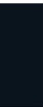
- bei Antwort des Servers umgekehrter Weg (Pseudo-Terminal fängt Antwort ab, leitet sie über TCP an den Client weiter, der die Ausgabe am Bildschirm macht)
- **Benutzername und Passwort werden unverschlüsselt übertragen**



## rlogin und rsh als Alternative zu Telnet (früher bei Unix)

---

- *rlogin* ist eine sehr flexible Alternative zu Telnet. Sogenannte Trusted Hosts können sich login-Name und Zugriffsrechte auf Dateien teilen.
- Vorteile gegenüber TELNET:
  - Bei rlogin auf einem Trusted Host entfällt die Abfrage des Passworts.
  - Da ausschließlich unter Unix verwendet, vereinfacht sich die Kommunikation zwischen Client und Server: beide Seiten kennen so etwas wie Standard Input und Output, Standard Error ...
  - Umgebungsvariablen des Benutzers (z.B. Terminaltyp) werden automatisch übertragen, so dass entfernte Sitzungen große Ähnlichkeit mit lokalen Sitzungen haben.
- *rsh* ist eine Variante von rlogin:
  - Ziel: Auf einfache Art und Weise einzelne Kommandos auf der Remote Machine ausführen ( `rsh machine command` ).
  - Automatische Authentifizierung erlaubt die Benutzung nicht nur interaktiv, sondern auch aus Programmen heraus (ohne Passwortabfrage).

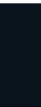




# Das ssh-Protokoll

---

- **ssh** adressiert die Sicherheitsprobleme von telnet und rlogin. Es ist ein Protokoll zur Erstellung einer sicheren Verbindung zwischen zwei Systemen. Alle während der Verbindung gesendeten und empfangenen Daten werden mit einer 128 Bit-Verschlüsselung verschlüsselt.
- ssh unterstützt verschiedene Authentisierungsarten:
  - Bei der so genannten hostbased-Authentifizierung akzeptiert ein Rechner ohne eigene account-spezifische Tests die Vorgaben eines fremden Rechners. Es wird höchstens die Identität des fremden Rechners überprüft
  - Die Authentifikation mit einem Passwort ist derzeit eine weitere Methode, um sich an einem Rechner anzumelden. Das Passwort wird verschlüsselt übertragen
  - Um auch das Übertragen eines verschlüsselten Passwortes zu vermeiden, wird ein public-key-Verfahren eingesetzt. Hierbei verbleibt der private key auf dem eigenen Rechner, der public-key wird auf dem Server hinterlegt. I.allg. muss man sich einmalig mit Benutzernamen und Passwort anmelden, um den public-key zu hinterlegen

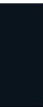




## SSH als Proxy: Port-Forwarding

---

- Port-Forwarding:
  - verschlüsselte Verbindung zwischen zwei beliebigen Ports
  - kann auch ohne Shell genutzt werden
  - lokaler Port führt direkt auf den Zielport, als wäre dieser lokal
- universell einsetzbar, u. a.:
  - FTP sicherer machen (Tunneln des Kommando-Ports)
  - POP3/SMTP (Mailversand) sicherer machen
  - X Window Datenverkehr absichern
- Aufruf:
  - `ssh -L port:zielHost:zielPort <Rechner>` leitet *localhost:port* via *Rechner* zu *zielHost:zielPort* weiter
  - `ssh -R port:zielHost:zielPort <Rechner>` leitet *Rechner:port* via *localhost* zu *zielHost:zielPort* weiter







# SNMP (Simple Network Management Protocol)

---

Die Objekte des Internet-Managements sind Rechner und vor allem Router

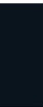
Ähnlich wie bei SMTP wird das Internet-Management durch zwei unabhängige standardisierte Teilbereiche beschrieben:

- Das Protokoll SNMP, das festlegt, wie Management-Information kommuniziert wird (Formate und Bedeutung von SNMP-Nachrichten) und
- die Spezifikation der Daten (MIB = *Management Information Base*).  
Die MIB spezifiziert die Informationseinheiten (*items*), die vorgehalten werden müssen, und welche Operationen darauf erlaubt sind.

Wie bei den anderen Anwendungsdiensten, funktioniert auch das Management nach dem *Client/Server*-Prinzip.

In jedem Objekt (vor allem Router) muss ein *Server* installiert sein, der die in der MIB spezifizierten Informationen sammelt, diese gegebenenfalls einem *Client* zur Verfügung stellt (per SNMP) und von einem *Client* Kommandos entgegennimmt.

Für das Ausführen von Management-Funktionen ist eine Authentifizierung erforderlich ist.





# SNMP (Simple Network Management Protocol)

Verwaltete Ressourcen integrieren **SNMP-Agenten** (Software-Prozess)

Die Agenten verwalten die Managementinformationen der Komponente

z.B. Anzahl eingegangener/verlorener Pakete

Der **Manager** (Software-Prozess) dient der Kommunikation mit den Agenten

Protokoll: SNMP (verwendet UDP)

